

Problem 1

5.

left = 1

right = K

result = 0

while left <= right

 totalDays = 0

 mid = (left + right) / 2

 for i = 1 to n

 totalDays += (piles[i] + mid - 1) / mid

 if totalDays > h

 left = mid + 1

 else

 right = mid - 1

 result = mid

return result

By using binary search to search from 1 to K, we can achieve a time complexity of $O(\log K)$ for the searching process. Noted that since the optimal k will never be larger than K, we do not have to consider $k > K$. In the while loop, we use a for loop from 1 to n to find the sum of the days taken given k, which is the "mid" in this context. The days spent on a single pile is calculated by $\lceil \frac{\text{piles}[i] + \text{mid} - 1}{2} \rceil$, which gives us the ceiling of the result. Since the total days taken will decrease as we increase k, the number of snacks eaten every day, we will cut off the lower portion when "totalDays" exceeds h, the limit, and cut off the upper portion vice versa. Then we store the value in result and return it in the end. Since all the other operations inside the loops are of $O(1)$ time complexity, the while loop is of $O(\log K)$, and the for loop is of $O(n)$ time complexity, we can achieve an overall time complexity of $O(n \log K)$.

Problem 2

1.

linkedList-Insertion-Sort(head)

if head == NIL or head.next == NIL

return head

dummy = new Node()

dummy.next = head

lastSorted = head

curr = head.next

while curr != NIL

if lastSorted.value <= curr.value

lastSorted = lastSorted.next

else

comp = dummy

lastSorted.next = curr.next

while curr.value > comp.next.value

comp = comp.next

curr.next = comp.next

comp.next = curr

curr = lastSorted.next

return dummy.next

2.

findDiffNum(array[], m)

check = 0

for i = 1 to 2m - 1

```
check = check ^ array[i]
```

```
return check
```

Since every integer from 1 to m appears twice, we can use XOR to cancel them out. The operation order does not matter in XOR, so we initialize "check" to 0 and XOR it with every element in the array. Since the elements that appear twice cancel each other out, the one that's left in the end is the element that only appears once. This algorithm only creates an int variable, taking $O(1)$ space, and it only scan through the array once with only the XOR operation, taking $O(m)$ time.

3.

```
linkedList-reorder(head)
```

```
slow = head
```

```
fast = head
```

```
prevSlow = NIL
```

```
while(fast.next != NIL)
```

```
    prevSlow = slow
```

```
    slow = slow.next
```

```
    if(fast.next.next != NIL)
```

```
        fast = fast.next.next
```

```
    else
```

```
        fast = fast.next
```

```
prevSlow.next = NIL
```

```
prev = NIL
```

```
curr = slow
```

```
while(curr != NIL)
```

```

    next = curr.next

    curr.next = prev

    prev = curr

    curr = next

currLeft = head
currRight = prev
while(currLeft != NIL && currRight != NIL)
    nextLeft = currLeft.next
    nextRight = currRight.next

    currLeft.next = currRight
    currLeft = nextLeft

    currRight.next = currLeft
    currRight = nextRight

return head

```

For the first chunk (finding the middle), since the fast pointer moves twice as fast as the slow pointer, so when the fast pointer reaches the end, the slow one will be at the middle. Then in the second chunk, the algorithm reverses the second half of the list and sever the it from the first half. The last chunk links the nodes across the two halves, reordering the nodes as expected. For a linked list with $2m$ nodes, finding the middle takes m steps as the fast pointer travels $2m$ nodes. Reversing the second half takes m steps. Merging the two halves takes $2m$ steps. Hence, the total time complexity is $O(m) + O(m) + O(m) = O(m)$. As for the space complexity, this algorithm only set up a fixed number of pointers without dynamically allocating memories, so the space complexity remains $O(1)$.

4.

In this context, 6, 7, 42 will be used to represent the multiple of 6, 7, 42, respectively, since the logic and the result

will not be affected. The pseudocode is wrong. For instance, if the input array is [6, 7, 42], there are $m = 2$ 6s and $k = 2$ 7s, the valid output will be [6, 42, 7], which places the 6s in the front and 7s in the back, making 42s in the middle. Yet, the output of this program will be [42, 6, 7] instead of [6, 42, 7]. The reason is that after the first loop, the program correctly places 7s to the back of the array, but in the second loop, the program incorrectly moves 42s to the front of the array, leaving 6s in the middle, while the output should be the opposite. This program will work only when there's no 42s in the input array.

5.

matrix_rotation(head)

//find the head of every columns

colHeadNodes[n]

colCurrNodes[n]

curr = head

for col = 1 to n

colHeadNodes[col] = curr

colCurrNodes[col] = curr

curr = curr.next

//link by columns

while curr != NIL

for col = 1 to n

colCurrNodes[col].next = curr

colCurrNodes[col] = curr

curr = curr.next

```

//link the last node of every column to the head of the previous column

for col = n to 2

    colCurrNodes[col].next = colHeadNodes[col - 1]

colCurrNodes[1].next = NIL

//reallocate the head of the list

head = colHeadNodes[n]

return head

```

A 90-degree counter-clockwise rotation transforms the first row to the first column, and the last column to the first row. First we set the every node in the first row to be the head of each column, then we connect the nodes in the same column. The final step is to connect the last node in every column to the first node of the previous column. By changing the direction of the links, the algorithm can mimic the rotation transformation. For time complexity, the algorithm checks every node in the list once, taking $O(mn)$ time. Then it connect the last node in a column to the first node in the previous column. This step takes $O(n)$ time since it only traverses through the last row of the original matrix. The overall time complexity is $O(mn) + O(n) = O(mn)$. For space complexity, the algorithm only dynamically allocates two arrays of size n to store the head and the current node of each column, resulting in a space complexity of $O(n)$.